

Python File Handling

Reading, Writing, the with Statement, and Exception Handling

1. Introduction

Most real programs need to persist data beyond a single run, or process data that already exists on disk — configuration files, logs, CSV exports, or user-generated documents. Python's built-in file handling tools make reading and writing files straightforward. This module also covers exception handling with `try/except`, since file operations are one of the most common sources of runtime errors (a file might not exist, permissions might be wrong, or the disk might be full).

By combining safe file handling with proper exception handling, you can write programs that gracefully deal with missing files, bad data, and other unexpected conditions instead of crashing outright.

2. Learning Objectives

- Open, read, write, and append to files using `open()` and file modes.
- Use the `with` statement to manage files safely and automatically close them.
- Understand file paths (absolute vs relative) and text encoding.
- Catch and handle exceptions with `try/except`, including `else` and `finally` clauses.
- Recognize and handle `ZeroDivisionError`, `ValueError`, and `FileNotFoundError` specifically.
- Write robust programs that fail gracefully instead of crashing.

3. Concept Explanations

3.1 Opening Files with `open()`

The built-in `open()` function opens a file and returns a file object you can read from or write to. The second argument specifies the **mode**.

Mode	Meaning
'r'	Read (default). Raises <code>FileNotFoundError</code> if the file does not exist.
'w'	Write. Creates the file if missing, or overwrites/truncates it if it exists.
'a'	Append. Creates the file if missing; writes are added to the end.
'x'	Exclusive creation. Fails if the file already exists.
'b'	Binary mode modifier, e.g. <code>'rb'</code> for reading binary data.

3.2 Reading Files

```
file = open("notes.txt", "r")
content = file.read()           # read entire file as a string
```

```

file.close()                                # must close manually without 'with'

with open("notes.txt", "r") as file:
    for line in file:                        # iterate line by line (memory-efficient)
        print(line.strip())                 # strip() removes the trailing newline

with open("notes.txt", "r") as file:
    lines = file.readlines()                # list of all lines, including '\n'

```

3.3 Writing and Appending Files

```

with open("output.txt", "w") as file:
    file.write("First line\n")
    file.write("Second line\n")           # 'w' mode overwrites existing content

with open("output.txt", "a") as file:
    file.write("Appended line\n")         # 'a' mode adds to the end, keeps existing content

```

3.4 The with Statement

The `with` statement creates a context that automatically closes the file when the block ends — even if an exception occurs inside it. This is strongly preferred over manually calling `.close()`, because it is easy to forget to close a file, or to skip closing it if an error is raised before the `close()` call is reached.

3.5 File Paths and Encoding

A file path can be **relative** (interpreted relative to the current working directory, e.g. `"data/notes.txt"`) or **absolute** (a full path from the root of the filesystem, e.g. `"/home/user/notes.txt"`). Text files also have an **encoding** — the scheme used to convert bytes on disk into characters. UTF-8 is the modern standard and should almost always be specified explicitly for portability.

```

with open("notes.txt", "r", encoding="utf-8") as file:
    content = file.read()

from pathlib import Path
path = Path("data") / "notes.txt"         # cross-platform path joining
print(path.exists())

```

3.6 try / except and Common Exceptions

A `try` block contains code that might raise an exception; the matching `except` block(s) handle specific error types if they occur, preventing the program from crashing. You can target specific exception types (recommended) rather than catching every possible error generically.

Exception	Typical Cause
<code>ZeroDivisionError</code>	Dividing a number by zero, e.g. <code>10 / 0</code>
<code>ValueError</code>	Converting an inappropriate value, e.g. <code>int("abc")</code>
<code>FileNotFoundError</code>	Opening a file in read mode that does not exist on disk

```

try:
    number = int(input("Enter a number: "))
    result = 100 / number

```

```

except ValueError:
    print("That was not a valid number.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
else:
    print(f"Result: {result}")      # runs only if no exception occurred
finally:
    print("Done attempting the calculation.") # always runs

```

3.7 else and finally

- **else:** runs only if the try block completed without raising an exception. Useful for code that should only run on success.
- **finally:** always runs, whether or not an exception occurred — ideal for cleanup code such as closing files or releasing resources.

```

try:
    with open("missing_file.txt", "r") as file:
        data = file.read()
except FileNotFoundError:
    print("File not found. Creating a new one.")
    with open("missing_file.txt", "w") as file:
        file.write("")
finally:
    print("File check complete.")

```

4. Syntax Summary

```

with open(path, mode, encoding="utf-8") as file:
    ...

try:
    risky_code()
except SpecificError as e:
    handle_it(e)
except AnotherError:
    handle_that()
else:
    only_if_no_error()
finally:
    always_runs()

```

5. Code Examples

Example 1: Write, Then Read a File

```

with open("log.txt", "w", encoding="utf-8") as file:
    file.write("Program started\n")
    file.write("Processing data\n")

with open("log.txt", "r", encoding="utf-8") as file:
    print(file.read())

```

Example 2: Safe Division with Full Exception Handling

```
def safe_divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return "Error: cannot divide by zero"

print(safe_divide(10, 2))
print(safe_divide(10, 0))
```

Example 3: Robust File Reader

```
def read_config(path):
    try:
        with open(path, "r", encoding="utf-8") as file:
            return file.read()
    except FileNotFoundError:
        print(f"{path} not found, using default configuration.")
        return ""
    finally:
        print("Config load attempt finished.")

config = read_config("settings.cfg")
```

6. Step-by-Step Explanation

Walking through Example 3 (Robust File Reader) line by line:

- 1 The function attempts to open and read the file inside a `with` block nested in a `try` block.
- 2 If the file exists, its contents are returned immediately, and the `finally` block still runs before the function returns.
- 3 If the file does not exist, Python raises `FileNotFoundError`, which is caught by the matching `except` clause instead of crashing the program.
- 4 Inside the `except` block, a friendly message is printed and an empty string is returned as a safe fallback value.
- 5 Regardless of which path was taken, the `finally` block executes, confirming the attempt is over.

7. Common Mistakes

■ **Common Mistake:** forgetting to close a file opened without `with`, which can leave the file locked or cause data loss on some systems.

■ **Common Mistake:** opening a file in `'w'` mode when you meant to append, accidentally erasing existing content.

■ **Common Mistake:** catching a bare `except: for everything`, which hides real bugs and makes debugging much harder.

■ **Common Mistake:** not specifying an encoding, which can cause `UnicodeDecodeError` on files containing non-ASCII characters.

■ **Common Mistake:** assuming a file exists and skipping error handling entirely, resulting in an unhandled `FileNotFoundError` crash.

8. Best Practices

✓ **Best Practice:** always use the `with` statement for file operations so files are closed automatically.

✓ **Best Practice:** catch specific exception types instead of a bare `except`, and only catch what you can meaningfully handle.

✓ **Best Practice:** always specify `encoding='utf-8'` explicitly when opening text files for portability.

✓ **Best Practice:** use `finally` (or `with`, which does this automatically) for cleanup actions like closing files or connections.

9. Practice Questions

- 1 What is the difference between 'w' mode and 'a' mode when opening a file?
- 2 Why is the `with` statement preferred over manually calling `open()` and `close()`?
- 3 What exception is raised when you try to open a non-existent file in read mode?
- 4 In what order do `try`, `except`, `else`, and `finally` execute when no exception occurs?
- 5 How would you convert a relative file path to an absolute one using `pathlib`?

10. Mini Coding Exercises

- 1 Write a program that reads a text file and counts the number of lines, words, and characters.
- 2 Write a program that appends a new to-do item to a file each time it runs.
- 3 Write a program that safely divides two numbers entered by the user, handling both `ValueError` and `ZeroDivisionError`.
- 4 Write a program that tries to open a file, and if it does not exist, creates it with default content instead.
- 5 Write a program that copies the contents of one text file into another using `with` statements.

11. Interview / QC Questions

- 1 What does the `with` statement do 'under the hood', in terms of the context manager protocol (`__enter__` and `__exit__`)?
- 2 Why is catching specific exceptions considered better practice than a bare `except` clause?
- 3 What is the difference between the `else` and `finally` clauses in a `try` statement?
- 4 How would you handle a situation where a file might be encoded in a format other than UTF-8?
- 5 Explain what happens if an exception is raised inside a `finally` block itself.

12. Summary

This module covered how to read from and write to files using `open()` and the various file modes, and why the `with` statement is the safe, idiomatic way to manage file resources in Python. You learned the difference between relative and absolute paths, the importance of specifying text encoding, and how `try/except/else/finally` lets a program handle predictable failure cases — such as `ZeroDivisionError`, `ValueError`, and `FileNotFoundError` — gracefully instead of crashing. Combining careful file handling with targeted exception handling is essential for writing programs that behave reliably in the real world, where input data and the filesystem are never perfectly predictable.

End of Python_File_Handling — continue to Python_Modules.pdf for the final module.